

SE_praxis

Disclaimer

Zur Erstellung dieses Dokuments wurde sehr viel mit LLMs gearbeitet. Die Informationen können fehlerhaft und unvollständig sein, deshalb unbedingt schon mit einer Grundahnung an die Sache herangehen und nicht ausschließlich die Zusammenfassung zum Lernen, sondern viel eher zum schnellen Wiederholen verwenden!

Inhalt:

1. [ue3](#)
2. [ue4](#)

ue3

1. Requirements Engineering

Ziel

Verstehen, was das System tun soll ("Was", nicht "Wie") und die Anforderungen sauber spezifizieren.

Functional vs. Non-functional Requirements

Hier musst du unterscheiden können, ob eine Anforderung eine Funktion beschreibt oder eine Eigenschaft des Systems.

- **Functional Requirements (Funktionale Anforderungen)**
 - Beschreiben das Verhalten des Systems (Input → Output).
 - Was *macht* die Software?
 - Beispiele:
 - "Das System berechnet die Steuer basierend auf dem Einkommen."
 - "Der Benutzer kann ein Buch ausleihen."
 - Reaktion auf Fehler/abnormale Situationen.
 - Interaktion mit externen Interfaces (Hardware, andere Software).
- **Non-functional Requirements (Nicht-funktionale Anforderungen)**
 - Beschreiben Qualitätsmerkmale oder Einschränkungen.
 - *Wie* macht es das System? (Performance, Security, etc.)
 - Kategorien:
 - **Performance:** Geschwindigkeit, Antwortzeit (Response Time), Verfügbarkeit.
 - **Design Constraints:** Standards, Betriebssystem, Programmiersprache (oft durch Client vorgegeben).
 - **Qualität:** Wartbarkeit (Maintainability), Portabilität, Sicherheit.

Praxis-Check

- "Der Button muss rot sein" → Design Constraint (Non-functional).
- "Wenn man auf den Button klickt, wird die Bestellung abgeschickt" → Functional.
- "Das Abschicken darf maximal 2 Sekunden dauern" → Performance (Non-functional).

Qualität von Requirements (Bad Requirements erkennen)

In der Prüfung musst du oft erkennen, warum ein Requirement "schlecht" ist und wie man es verbessert.

Kriterien für gute Requirements:

1. **Correctness:** Spiegelt den Wunsch des Kunden wider.
2. **Consistency:** Keine Widersprüche (z.B. Req A sagt "max 2s", Req B sagt "Echtzeit").
3. **Completeness:** Alle Szenarien (auch Fehlerfälle) abgedeckt.
4. **Clarity (Unambiguity):** Nur **eine** mögliche Interpretation.
5. **Realism:** Muss umsetzbar sein.
6. **Verifiability:** Man muss einen Test schreiben können, der beweist, dass das Requirement erfüllt ist.

Typische Fehler & Lösungen:

Schlechtes Requirement	Problem	Bessere Lösung
"Das System soll von älteren Menschen nutzbar sein."	Unclear / Not verifiable. "Nutzbar" ist subjektiv.	"Text muss mindestens 1cm groß sein."
"Das Produkt soll fehlerfrei sein."	Not realistic / Not verifiable. Software hat immer Bugs.	Testkriterien spezifizieren (z.B. "Besteht 100% der Unit Tests").
"Das System soll Echtzeit-Antworten liefern."	Unclear. Was heißt Echtzeit?	"Das System antwortet in weniger als 2s."

Actors, Scenarios & Use Cases

Erkennen von Elementen aus einem Text (Requirements Elicitation).

- **Actors:** Rollen, die mit dem System interagieren (Nutzer, Admins, aber auch externe Systeme/Hardware).
 - **Frage:** Wer nutzt die Hauptfunktionen? Wer wartet das System?
- **Scenario:** Eine konkrete Beschreibung (Instanz), was **ein** Nutzer in einer spezifischen Situation tut.
 - **Fokus:** Verständlichkeit.
 - **Beispiel:** "Alice scannt ihren Ausweis, dann das Buch. Das System verbucht es."
- **Use Case:** Verallgemeinerung von Szenarien. Beschreibt alle möglichen Fälle (Schritte) um ein Ziel zu erreichen.
 - **Fokus:** Vollständigkeit.
 - **Beispiel:** Tabelle mit "Actor Steps" (Scannt Karte) vs. "System Steps" (Validiert Karte).

Example: **Actor steps**

1. Scans library card

3. Selects 'Borrow'

5. Scans barcode of book to be borrowed

System steps

2. Validates card; returns card; displays user data; displays 'Select Function' dialog

4. Displays 'Borrow' dialog

6. Identifies book; records book as borrowed...

2. Dafny & Formale Modellierung

Design by Contract

Wir definieren präzise Schnittstellen mittels **Preconditions** (Was erwartet der Code?) und **Postconditions** (Was garantiert er?).

Methoden vs. Funktionen in Dafny

Ein häufiger Fehler ist die Verwechslung dieser beiden Konstrukte.

- **Method** (`method`):
 - Imperativ (wie in Java/C#).
 - Kann State ändern (Heap, Arrays).
 - Kann **nicht** direkt in Spezifikationen (z.B. in einem `assert` oder `requires`) verwendet werden (außer sie ist rein, was selten ist).
 - Hat `requires` (Pre) und `ensures` (Post).
 - Syntax: `method Triple(x: int) returns (r: int) { ... }`
- **Function** (`function`):
 - Ein mathematischer Ausdruck / Value.
 - **Deterministisch**: Gleiche Argumente → Gleiches Ergebnis.
 - **Side-effect free**: Kann nichts verändern (kein Heap-Write).
 - Hat keinen "Body" mit Statements, sondern ist ein Ausdruck (Expression).
 - Wird oft in Spezifikationen verwendet.
 - Syntax: `function Average(a: int, b: int): int { (a+b)/2 }`

Wichtig für die Prüfung

Functions haben keine `out-parameter`. Ihr Body ist ein einziger Ausdruck.

Pre- & Postconditions (Strong vs. Weak)

Hier geht es oft um Vererbung (LSP - Liskov Substitution Principle) oder Interface-Design.

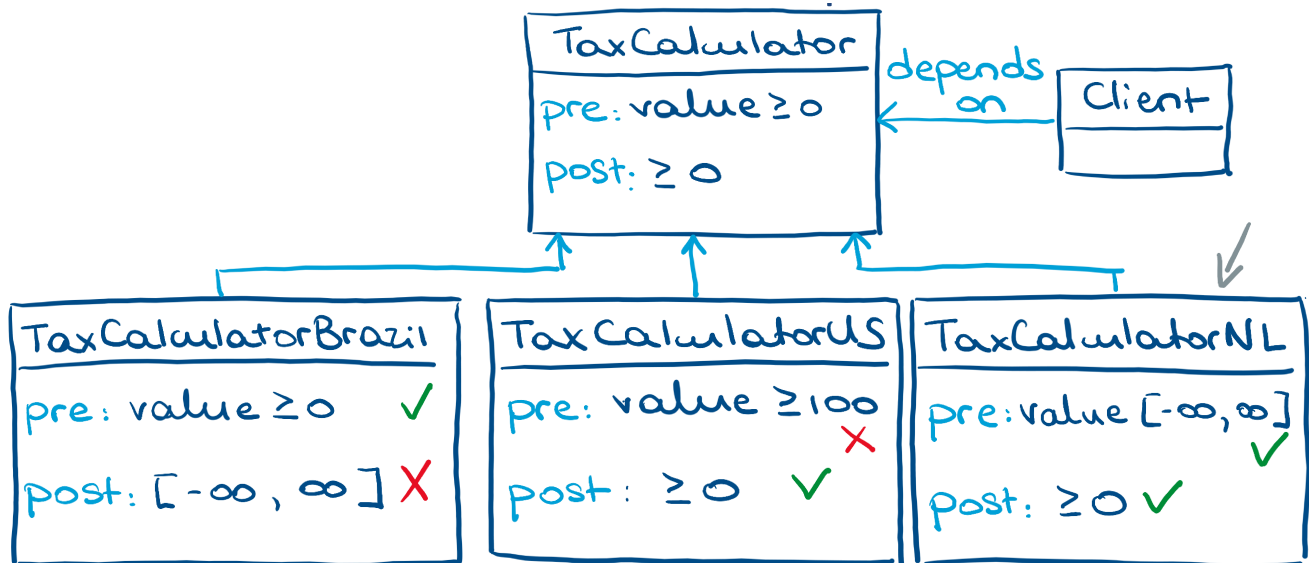
- **Precondition** (`requires`): Anforderung an den Aufrufer (Caller).
 - **Weak Precondition**: Akzeptiert **viele** Werte (z.B. `x: int`). Gut für den Caller, schwer für die Methode (muss alles behandeln).
 - **Strong Precondition**: Akzeptiert **wenige** Werte (z.B. `x > 0 && x < 10`). Gut für die Methode (kann Annahmen treffen), schwer für den Caller.
 - **Vergleich**: $x > 5$ ist **stärker** als $x > 0$ (weil es weniger Werte zulässt).
- **Postcondition** (`ensures`): Garantie für den Aufrufer.

- **Strong Postcondition:** Garantiert *viel* (präzises Ergebnis, z.B. `result == x * 2`). Gut für Caller.
- **Weak Postcondition:** Garantiert *wenig* (z.B. `result > 0`).

Liskov Substitution Principle (Vererbung):

Wenn eine Klasse `S` von `B` erbt:

1. Preconditions in `S` müssen **gleich oder schwächer** sein ($\leq$). (Man darf nicht strenger werden, sonst bricht der Code des Callers, der nur `B` kennt).
2. Postconditions in `S` müssen **gleich oder stärker** sein ($\geq$). (Man muss mindestens das liefern, was `B` versprochen hat).



Ghost Functions / Code

- **Definition:** Code, der nur für die Spezifikation/Verifikation existiert.
- **Verwendung:** `assert`, `requires`, `ensures`, Hilfsvariablen.
- **Compiler:** Der Compiler löscht Ghost-Code. Er landet nicht im ausführbaren Programm.
- **Zweck:** Ermöglicht Reasoning über Zustände, die im normalen Code zu teuer oder unnötig zu berechnen wären.

Korrektheit: Partiell vs. Total

- **Partielle Korrektheit:** *Falls* das Programm terminiert, ist das Ergebnis korrekt (erfüllt Postcondition).
- **Totale Korrektheit:** Partiell korrekt + **es terminiert immer**.
- Dafny versucht standardmäßig, totale Korrektheit zu beweisen.

3. Dafny: Loops & Termination

Um totale Korrektheit bei Schleifen oder Rekursion zu beweisen, müssen wir zeigen, dass sie enden.

Ranking Functions (Decreases)

Dafny braucht einen Beweis, dass eine Schleife/Rekursion endet.

- Man gibt einen Ausdruck an (`decreases ...`), der:
 1. In einer "well-founded relation" ist (hat ein Minimum, geht nicht unendlich nach unten, z.B. Natürliche Zahlen 0).
 2. Bei jedem Schritt **strikt kleiner** wird.
- Beispiel: `decreases n - i` in einer Schleife, wo `i` hochzählt bis `n`.

Lexikographische Tupel

Manchmal reicht eine Variable nicht für den Termination-Beweis (z.B. Ackermann-Funktion oder verschachtelte Loops).

- Format: `(a, b, c)`
- **Lesart:** Man vergleicht von links nach rechts. Die linke Komponente ist am "signifikantesten".
- `(4, 12)` ist größer als `(4, 11)`.
- `(4, 2)` ist größer als `(3, 500000)` (weil $4 > 3$, egal was danach kommt).
- Dafny nutzt dies automatisch bei `decreases a, b`.

Loop Invariants

Schleifen sind für Verifier schwer, weil man nicht jeden Durchlauf simulieren kann. Man nutzt Invarianten.

- **Definition:** Eine Bedingung, die **immer** wahr ist:
 1. Vor der Schleife (Pre-State).
 2. Vor jedem Schleifen-Check.
 3. Nach jedem Schleifen-Durchlauf.
- **Ziel:** Wenn die Schleife endet (Guard ist `false`), gilt: `Invariant && !Guard` → das muss reichen, um die Postcondition der Methode zu beweisen.
- **Typisches Muster:** Wenn ein Array `a` der Länge `n` durchlaufen wird mit Index `i`:
 - `invariant 0 <= i <= n` (Wichtig: `<=` n, weil i am Ende n ist).

4. Dafny: Arrays & Heaps

Arrays

- Sind Referenztypen (`array<int>`).
- Länge ist fix nach Allokation: `a.Length`.
- Zugriff: `a[i]`. Dafny verlangt Beweis, dass `0 <= i < a.Length`.

Reads & Modifies Clauses (Frames)

Dafny muss wissen, was eine Methode/Funktion im Heap lesen oder schreiben darf (Side Effects).

- **modifies:** Für Methoden. Gibt an, welche Objekte/Arrays verändert werden dürfen (Write Frame).

- Wenn eine Methode ein Array ändert, muss `modifies a` in der Spec stehen.
- Alles, was nicht in `modifies` steht, bleibt unverändert (garantiert).
- **reads:** Für Funktionen. Gibt an, von welchen Heap-Objekten das Ergebnis abhängt.
 - Nötig, damit Dafny weiß, wann sich der Wert der Funktion ändern könnte.

⚠ Fresh Values

Arrays/Objekte, die *innerhalb* der Methode neu erstellt (`new`) werden, müssen nicht in `modifies` stehen, da sie vorher nicht existierten (sie sind "fresh").

5. Testing & Verification Failures

📖 Der Testing Pyramid

Unit Tests (unten, viele) → Integration Tests → System Tests (oben, wenige, teuer).

Failures in Verification vs. Validation

- **Verification (Dafny):** "Are we building the product right?" (Mathematischer Beweis gegen Spezifikation).
 - *Failure:* Spezifikation ist nicht erfüllbar, Code verletzt Invariante, Termination nicht beweisbar.
- **Validation (Testing):** "Are we building the right product?" (Erfüllt es die Kundenwünsche?).
 - *Failure:* Tests schlagen fehl, Requirement falsch verstanden.

Integration Testing & System Testing

- **System Testing:** Testet das **ganze** System von außen (Black Box).
 - Egal wie es innen funktioniert.
 - Input X → Output Y.
 - Beispiel: Klick durch eine Web-App (End-to-End).
- **Integration Testing:** Testet das Zusammenspiel zwischen eigenem Code und externen Teilen (Datenbank, Frameworks, andere Klassen).
 - Beispiel: Testet, ob `Class A` korrekt `Class B` aufruft oder Daten aus der DB liest.

Mutation Testing

Ein Weg, die Qualität der Tests zu messen.

1. Man baut absichtlich Fehler (Mutanten) in den Code ein (z.B. `>` statt `>=`).
2. Man lässt die Tests laufen.
3. **Mutant killed:** Tests schlagen fehl → Gut!
4. **Mutant survived:** Tests laufen durch → Schlecht (Lücke in der Testabdeckung).

Test Doubles (Mocks, Stubs, etc.) & Dependency Injection

Wenn wir `Class A` testen wollen, aber sie von `Class B` (z.B. Datenbank) abhängt, wollen wir `B` oft nicht wirklich nutzen (zu langsam, komplex). Wir nutzen Test Doubles.

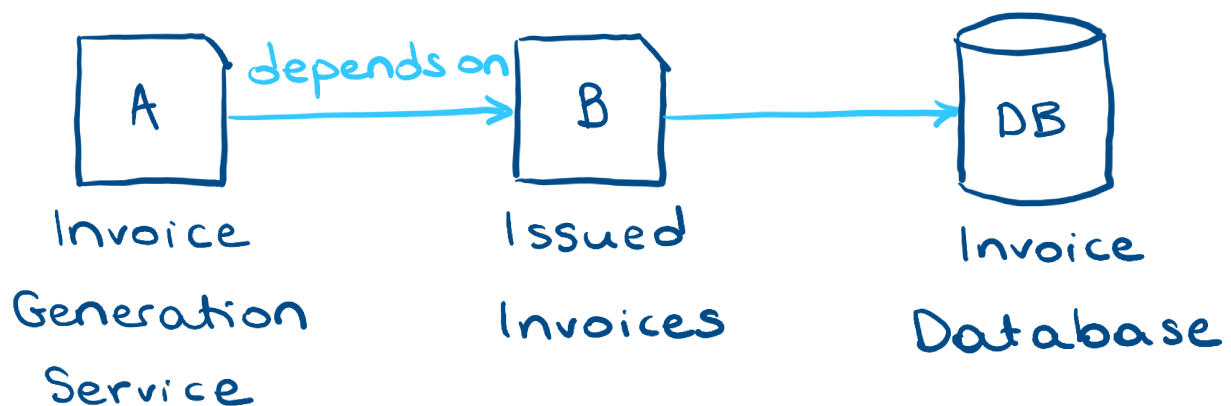
Dependency Injection (DI):

Damit wir Doubles nutzen können, darf `Class A` `Class B` nicht fest mit `new B()` erzeugen. Wir müssen `B` von außen "injizieren" (z.B. im Constructor übergeben).

- **Vorher (schwer testbar):** `class A { B b = new B(); ... }`
- **Nachher (DI):** `class A { B b; constructor(B dep) { this.b = dep; } ... }` → Im Test können wir nun einen Stub übergeben.

Arten von Test Doubles:

1. **Dummy:** Wird übergeben, aber nie benutzt (nur um Parameterliste zu füllen).
2. **Fake:** Funktionierende, aber vereinfachte Implementierung (z.B. In-Memory Datenbank statt SQL).
3. **Stub:** Liefert **hard-coded** Antworten. ("Wenn Methode X aufgerufen wird, gib immer 5 zurück"). Prüft Zustand.
4. **Mock:** Erwartet spezifische Aufrufe. ("Methode X muss genau 1x mit Argument 'Test' aufgerufen werden"). Prüft Verhalten (Interaktion).
5. **Spy:** "Echter" Wrapper, der protokolliert, was aufgerufen wurde.



ue4

Übersicht: Software Testing für Übung 4

Diese Mitschrift verbindet die Theorie aus den Vorlesungen mit den praktischen Anforderungen der Übung 4 (Specification-based Testing, Structural Testing, Mutation Testing, Data Flow Coverage).

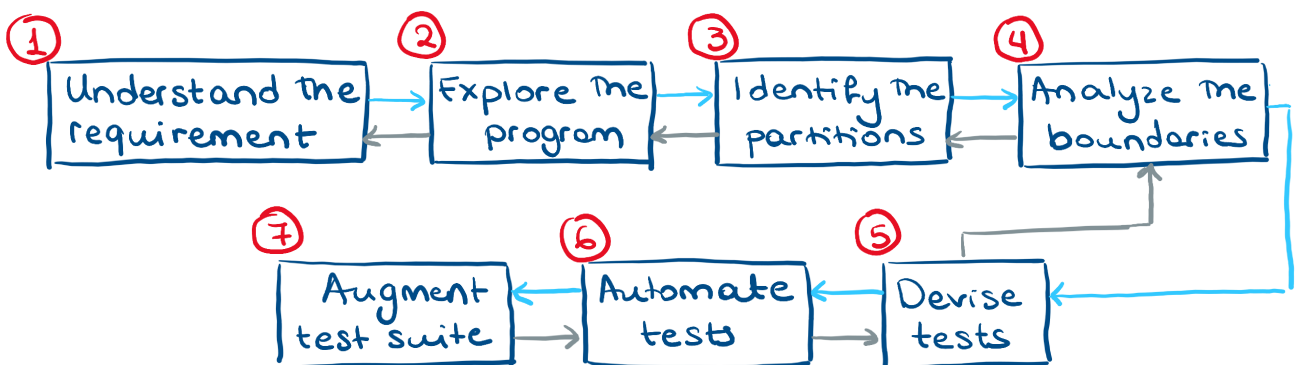
1. Specification-Based Testing (Black-Box)

📌 Relevanz für Übung 1

Diese Methode nutzt man, um Testfälle direkt aus den Anforderungen (Requirements) abzuleiten, ohne den Code zu kennen.

Der 7-Schritte-Ansatz

Um systematisch Tests abzuleiten, folge diesem Prozess:



— standard path to follow

— the process is iterative

1. **Understand the requirement:** Verstehe, was das Programm tun soll (Input/Output).
2. **Explore the program:** Spiele mit der Software (wenn möglich), um ein Gefühl dafür zu bekommen.
3. **Identify partitions:** Teile den Input-Raum in Äquivalenzklassen auf.
 - *Idee:* Alle Werte einer Partition sollten vom Programm gleich behandelt werden.
 - Man unterscheidet **Input-Partitionen** (was geht rein) und **Output-Partitionen** (was kommt raus).
4. **Analyze boundaries:** Fehler passieren oft an den Grenzen von Partitionen.
 - Identifiziere **On-Points** (Werte genau auf der Grenze).
 - Identifiziere **Off-Points** (Werte so nah wie möglich an der Grenze, aber auf der anderen Seite).

5. **Devise test cases:** Wähle konkrete Werte aus den Partitionen und Grenzen.
 - Kombiniere Partitionen sinnvoll (nicht alle Kombinationen sind möglich/nötig).
6. **Document expected outcome:** Lege fest, was passieren *muss* (Oracle).
7. **Augment the suite:** Füge Tests hinzu, die durch Erfahrung oder Intuition wichtig erscheinen könnten (die nicht strikt durch Partitionen abgedeckt sind).

Analyse von Boundaries (Grenzwertanalyse)

Besonders wichtig für Schleifen, Bedingungen (`<`, `<=`) und Arrays.

On-Point & Off-Point

- **On-Point:** Ein Wert, der *genau* auf der Grenze liegt.
- **Off-Point:** Ein Wert, der *unmittelbar* neben der Grenze liegt und das Ergebnis ändert (z.B. Partition wechselt).

Beispiel für Bedingung `x >= 10`:

- Grenze ist 10.
- **On-Point:** 10 (Bedingung wahr).
- **Off-Point:** 9 (Bedingung falsch, bei ganzen Zahlen).

2. Structural Testing (White-Box) & Coverage

Relevanz für Übung 2

Hier geht es darum, Tests basierend auf der internen Struktur des Codes zu erstellen.

Control Flow Graph (CFG)

Um Coverage zu messen, stellt man den Code als Graphen dar:

- **Knoten (Nodes):** Basic Blocks (Sequenz von Statements ohne Sprünge).
- **Kanten (Edges):** Kontrollfluss (Sprünge, if/else, loops).

Coverage Kriterien (Hierarchie)

Die Kriterien bauen aufeinander auf (Subsumption).

1. **Statement Coverage (Zeilenabdeckung):**
 - Jeder Knoten im CFG muss mindestens einmal besucht werden.
 - *Problem:* Übersieht oft fehlende `else`-Zweige oder leere Blöcke.
2. **Branch Coverage (Zweigabdeckung):**
 - Jede Kante (True/False bei `if`) muss abgelaufen werden.
 - Subsumiert Statement Coverage (meistens).

3. Path Coverage (Pfadabdeckung):

- Jeder mögliche Pfad durch das Programm wird getestet.
- **Problem:** Bei Schleifen oft unendlich viele Pfade → in der Praxis oft unmöglich ("infeasible").

4. Condition Coverage:

- Betrachtet atomare Bedingungen in zusammengesetzten Ausdrücken (z.B. `if (A || B)`).
- Jede atomare Bedingung (A, B) muss einmal `true` und einmal `false` sein.
- **Achtung:** Garantiert **nicht** Branch Coverage!

MC/DC (Modified Condition/Decision Coverage)

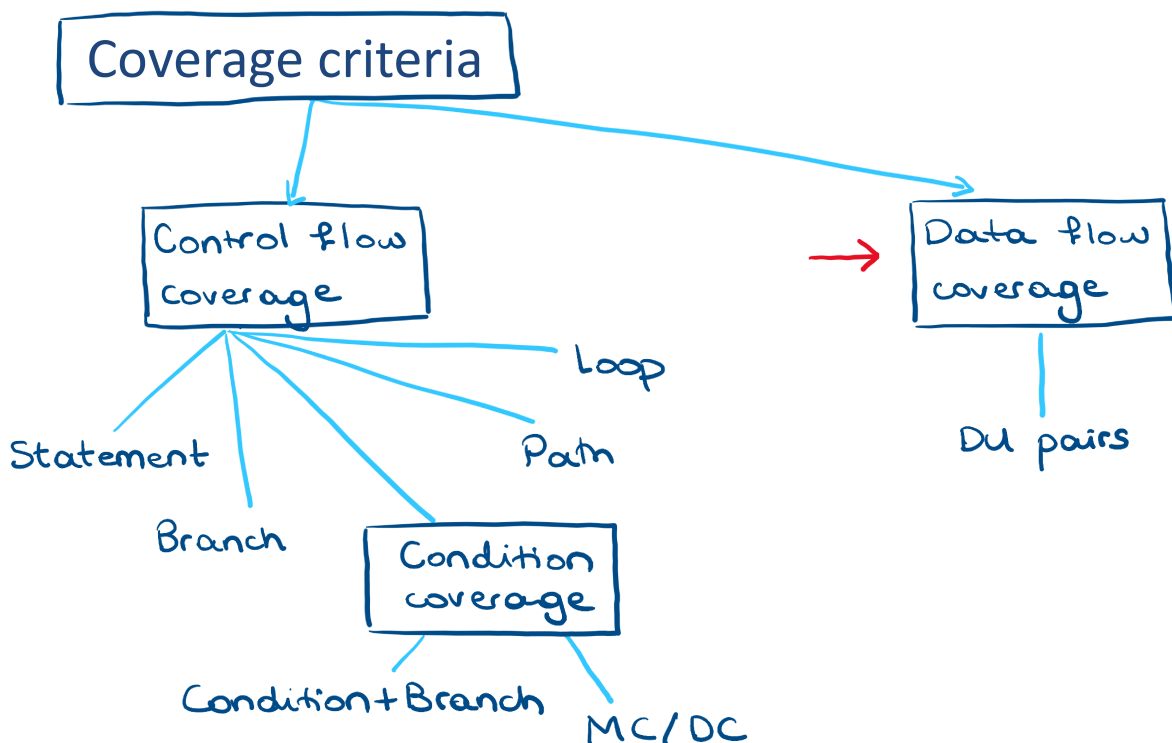
Wichtig für kritische Systeme (Luftfahrt etc.).

- Jede atomare Bedingung muss unabhängig das Ergebnis der Entscheidung beeinflussen.
- Erfordert:
 1. Jede Entscheidung nimmt alle möglichen Ergebnisse an.
 2. Jede Bedingung nimmt alle möglichen Ergebnisse an.
 3. Jede Bedingung wird gezeigt, dass sie das Ergebnis **unabhängig** beeinflusst (indem man die anderen fixiert).

Subsumption (Wer schlägt wen?)

Path Coverage $\supset$ MC/DC $\supset$ Condition + Branch Coverage $\supset$ Branch Coverage $\supset$ Statement Coverage.

(Hinweis: Condition Coverage alleine subsumiert Branch Coverage NICHT!)



3. Data Flow Coverage

Relevanz für Übung 4

Fokus auf Variablen: Wo werden sie definiert (geschrieben) und wo verwendet (gelesen)?

Definitionen

- **Variable Definition (*def*)**: Ein Statement, das einer Variable einen Wert zuweist (z.B. `x = 5`, `x++`, Input Parameter).
- **Variable Use (*use*)**: Ein Statement, das den Wert einer Variable liest.
 - **Computation Use (c-use)**: Berechnung, Output, Index (z.B. `y = x + 1`, `print(x)`).
 - **Predicate Use (p-use)**: In einer Bedingung (z.B. `if (x > 0)`).

Def-Clear Path & DU-Pairs

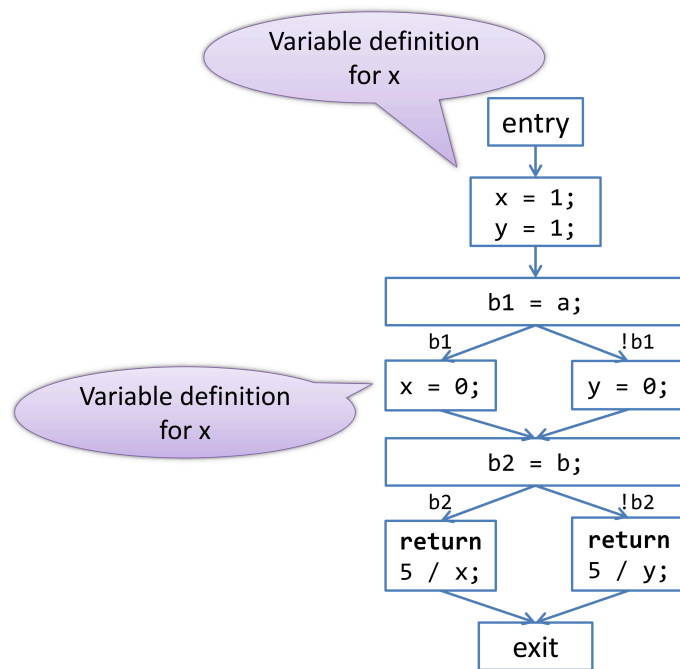
- **Definition-Clear Path**: Ein Pfad von einer Definition d zu einer Verwendung u , ohne dass die Variable dazwischen neu definiert (überschrieben) wird.
- **Def-Use Pair (du-pair)**: Ein Paar (d, u) , sodass es einen *definition-clear path* von d nach u gibt.

Vorgehensweise für Übung

1. Finde alle Stellen, wo Variable v gesetzt wird ($d_1, d_2, \dots$).
2. Finde alle Stellen, wo v gelesen wird ($u_1, u_2, \dots$).
3. Prüfe für jedes Paar (d_i, u_j) , ob es einen Weg gibt, auf dem v nicht verändert wird.
4. Ein Testfall "covert" ein du-pair, wenn er diesen Pfad ausführt.

Data Flow Kriterien

- **All-defs**: Für jede Definition d muss mindestens ein Pfad zu irgendeinem u getestet werden.
- **All-uses**: Für jede Definition d muss ein Pfad zu *jedem* erreichbaren u getestet werden.
- **All-du-paths**: Wie All-uses, aber wenn es mehrere schleifenfreie Pfade zwischen d und u gibt, müssen alle abgedeckt werden (oft sehr teuer).



4. Mutation Testing

Relevanz für Übung 3

Testet die Qualität der Testsuite, nicht des Codes.

Konzept

Man injiziert künstliche Fehler (**Mutanten**) in den Programmcode (z.B. `+` wird zu `-`, `>=` wird zu `>`).

- **Killed Mutant:** Ein Test schlägt fehl → Gut! Der Test hat den Fehler gefunden.
- **Survived Mutant:** Alle Tests laufen durch → Schlecht! Die Testsuite ist nicht stark genug, um diese Änderung zu bemerken.

Mutation Score

$$\text{Mutation Score} = \frac{\text{Killed Mutants}}{\text{Total Mutants} - \text{Equivalent Mutants}}$$

Equivalent Mutants

Ein Mutant, der semantisch identisch zum Original ist, obwohl der Syntax anders ist. Er *kann* von keinem Test getötet werden.

- **Beispiel:** Original `int x = a + b;` → Mutant: `int x = a + b;` (In Schleifenoptimierungen oder toten Code kann das vorkommen).
- Oder logisch: `if (x < 0)` geändert zu `if (x != 0)` ist äquivalent, WENN vor diesem Block sichergestellt wurde, dass `x` niemals positiv sein kann.

Vorgehen bei "Surviving Mutants"

Wenn ein Mutant überlebt, musst du analysieren warum:

1. Fehlt ein Testfall für diesen spezifischen Pfad/Wert?
2. Ist es ein äquivalenter Mutant? (Dann kann man ihn ignorieren/dokumentieren).

5. Test Doubles (Mocks & Stubs)

Relevanz: Theorie für Isolation von Tests (Unit Testing)

Wenn eine Klasse Abhängigkeiten hat (z.B. Datenbank, Webserver), nutzt man Doubles, um sie isoliert zu testen.

- **Dummy:** Objekte, die herumgereicht, aber nie verwendet werden (Parameterfüller).
- **Stub:** Liefert vorbereitete ("canned") Antworten auf Aufrufe zurück. ("Wenn `getBalance()` gerufen wird, gib `100` zurück").
- **Spy:** Ein Stub, der zusätzlich aufzeichnet, wie er aufgerufen wurde (z.B. "Wurde `send()` genau 1x gerufen?").
- **Mock:** Ein Objekt, das Erwartungen an Aufrufe definiert. Wenn diese nicht erfüllt werden (z.B. falsche Argumente), schlägt der Test fehl.
- **Fake:** Eine funktionierende Implementierung, aber vereinfacht (z.B. In-Memory Datenbank statt echter DB).

Wann Mocking nutzen?

- Abhängigkeit ist sehr langsam (DB, Netzwerk).
- Abhängigkeit ist schwer zu konfigurieren.
- Nicht-deterministisches Verhalten (Zufall, Zeit).
- Fehler schwer zu simulieren (z.B. "Simuliere Festplatte voll").

6. Testing Pyramide & Begriffe

Kontext

- **Unit Testing:** Testet einzelne Klassen/Methoden isoliert. Schnell, viele Tests. (Basis der Pyramide).
- **Integration Testing:** Testet das Zusammenspiel mehrerer Komponenten.
- **System Testing:** Testet das gesamte System (End-to-End). Langsam, weniger Tests. (Spitze der Pyramide).

- **Refactoring:** Code verbessern, ohne das Verhalten zu ändern. TDD (Test Driven Development) hilft hier sicherzustellen, dass nichts kaputt geht ("Red-Green-Refactor").

Testbarkeit

Code sollte "Design for Testability" aufweisen:

- Observability (Man kann Zustand/Output sehen).
- Controllability (Man kann Input/Zustand steuern).
- Isolation (Komponenten sind entkoppelt -> Dependency Injection).